Rapport Labo 1 - Computer Architectuur

Victor Deforce en Cían Olieslagers

February 16, 2025

1 Probleem

1.1 voorkennis

CUDA (Compute Unified Device Architecture) is een programmeermodel en API van NVIDIA waarmee ontwikkelaars de rekenkracht van een GPU kunnen benutten voor parallelle berekeningen. In plaats van één taak na elkaar uit te voeren (zoals een CPU), kan een GPU duizenden berekeningen tegelijk doen door gebruik te maken van threads.
Een thread is de kleinste eenheid van uitvoering in een GPU-programma. Threads worden gegroepeerd in blokken, en blokken vormen samen een grid. Elke thread krijgt een unieke index toegewezen en werkt op een specifiek deel van de data. Belangrijke parameters hierbij zijn:

1. threadIdx.x: de index van een thread binnen een blok
2. blockIdx.x: de index van een blok binnen het grid
3. blockDim.x: het aantal threads per blok
4. gridDim.x: het aantal blokken in het grid

1.2 Opgave

De oefening bestaat uit het omdraaien van een array met behulp van parallelle verwerking op de GPU. Dit gebeurt door meerdere threads, waarbij elk element van de array naar de juiste omgekeerde positie wordt verplaatst. De uitvoering gebeurt binnen 4 blocks met een aantal threads gelijk aan de grootte van de array. De code moet correct gebruik maken van geheugenallocatie, kernel-executie en data-overdracht tussen CPU en GPU om het verwachte resultaat te bekomen.

2 Eigen Implementatie

We initialiseren een array h_a met lengte arraySize als input en alloceren een tweede array h_c als output. We alloceren ook twee arrays, d_a en d_c, met dezelfde lengte in het geheugen van de GPU en kopiëren de input array h_a naar d_a.
De switchKernel wordt opgeroepen met 4 blokken en arraySize/4 threads per block. Verder geven we d_c, d_a en arraySize mee. Deze functie doet thread i uit block j dan element i + 4j uit d_c gelijk zetten aan arraySize - i - 4j - 1 uit d_a. In de kernel gebruiken we threadIdx.x, de index van een thread binnen een blok, om elk element correct te verwerken. De andere CUDA-parameters (blockIdx.x, blockDim.x en gridDim.x) zijn niet nodig omdat er slechts één blok wordt gebruikt.
We synchroniseren de threads om zeker te zijn dat ze allemaal voltooid zijn met *cudaDeviceSynchronize()*. En kopiëren dan d_c terug naar h_c. Ten slotte worden de gealloceerde stukken geheugen terug vrijgegeven.

3 Appendix

3.1 Code

```
#include <stdio.h>


__global__ void switchKernel(int *c, int arraySize,const int *a)
{
    int i = threadIdx.x;
    int j = blockIdx.x;
    c[arraySize - i - j * blockDim.x-1] = a[i + j * blockDim.x];

}

int main()
{
    const int arraySize = 8;
    const int h_a[arraySize] = { 1, 2, 3, 4, 5, 6, 7, 8 };
    int h_c[arraySize] = { 0 };

    int* d_a, * d_c;

    cudaMalloc((void**)&d_a, arraySize * sizeof(int));
    cudaMalloc((void**)&d_c, arraySize * sizeof(int));
    cudaMemcpy(d_a, h_a, arraySize * sizeof(int), cudaMemcpyHostToDevice);

    switchKernel<<< 4, arraySize/4 >>>(d_c, arraySize, d_a);


    cudaError_t error = cudaGetLastError();
    if (error != cudaSuccess) {
        printf("Probleem: %s\n", cudaGetErrorString(error));
    }

    cudaDeviceSynchronize();

    cudaMemcpy(h_c, d_c, arraySize * sizeof(int), cudaMemcpyDeviceToHost);

    for (int i = 0; i < arraySize; i++) {
        printf("%i ", h_c[i]);
    }

    cudaFree(d_c);
    cudaFree(d_a);

    return 0;
}
```